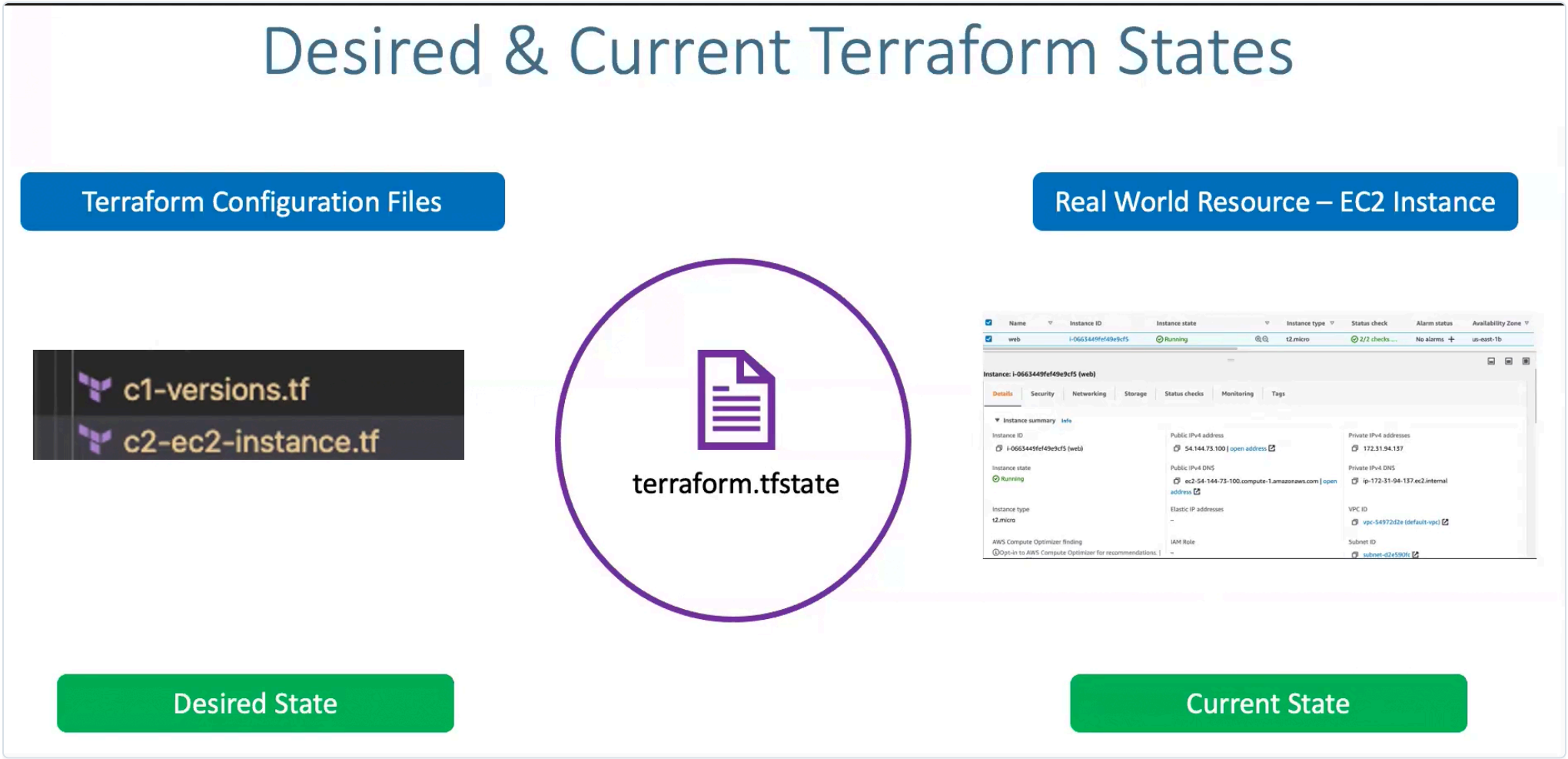
Screenshots — Terraform State Files, HCL Block Syntax & Resource References

Lecture 2 · 25 shots, chronological · source stills from the session (2026-09-22 22:42 – 2026-09-23 00:37)

01 / 25 Title slide — Desired & Current Terraform States (config files → terraform.tfstate → real EC2 resource) 22:42:04



What is State File ?

The state of a Terraform project is a file that stores all details about resources that were created in the context of a given project.

This state file contains information about the provisioned infrastructure which terraform manage. This is created and updated each time
an apply or destroy command is run.

"it basically act as a database for provisioned resources"
This state file is used by Terraform to map real world resources to our configuration, keep track of metadata, and to improve performance for large infrastructures.

Terraform store the state or info of the infrastructure that is being created by terraform files.
This state allows to map your real work infra with you terraform configuration file.



This state is stored by default in a local file named **terraform.tfstate**

Terraform uses this local state to create plans and make changes to our infrastructure. Before any terraform operation, Terraform does a **refresh** to update the state with the **real** infrastructure.

Desired State and Current State

Desired State=> "this is the state that is mentioned in your resource block of tf file."

It is the State of infrastructure resources as per the terraform file (Eg. ex1.tf).

03 / 25 Block syntax diagram — block_type / block_label / block_name (aws_vpc "vpc" example) 22:49:06

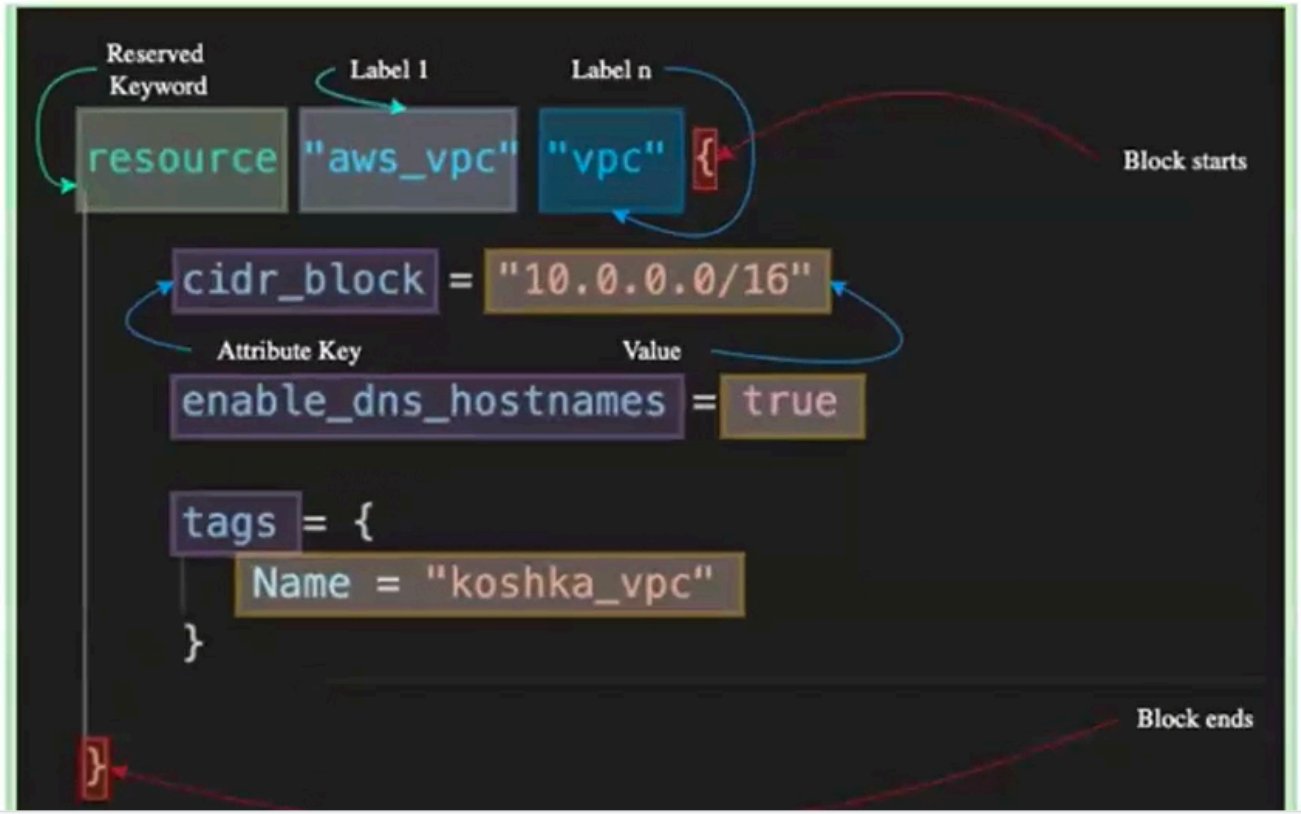
```
<block_type> "<block_label>" "<block_name>" {

  # Configuration settings for the resource

  attribute1 = value1

  attribute2 = value2

}
```

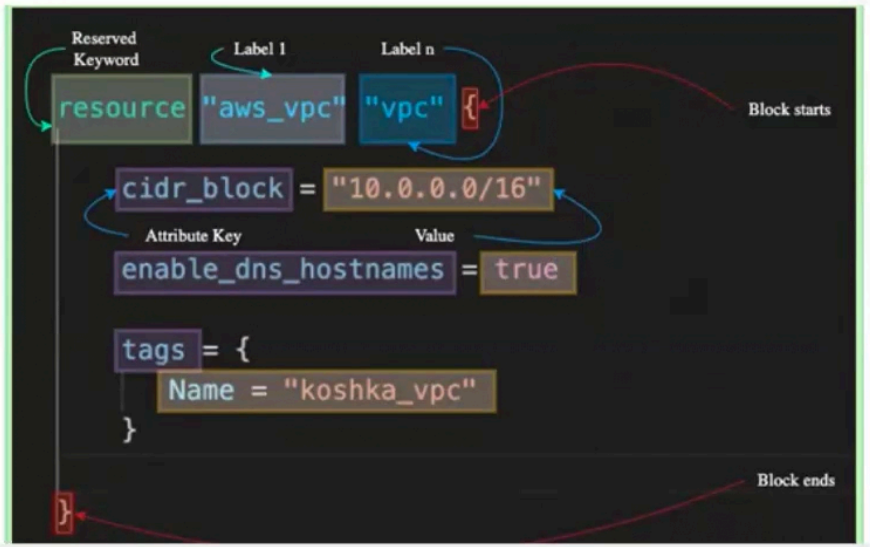


04 / 25 Declarative language + common block types (provider, resource, variable, output, module) with aws_s3_bucket example 22:58:47

```
<block_type> "<block_label>" "<block_name>" {
# Configuration settings for the resource
attribute1 = value1
attribute2 = value2
}
```

```
resource "aws_s3_bucket" "first" {
  bucket = "linux23212google123"
  tags = {
    Name = "my_buck"
  }
}
```

declarative language



A **block** in Terraform is like a **section** of code that tells Terraform what to do.

Common block types:

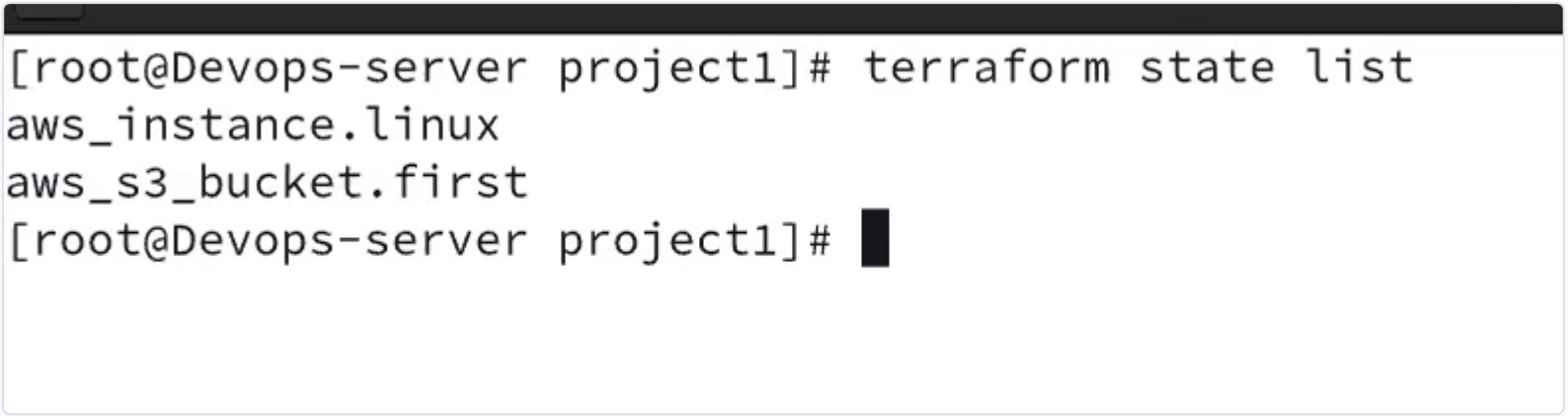
- provider – configures a provider (e.g., AWS, Azure).
- resource – defines a resource to be created.
- variable – declares input variables.
- output – defines outputs from a module.
- module – includes external Terrafo

05 / 25 Terminal — `nl main.tf`: EC2 instance + S3 bucket resource blocks 23:06:34

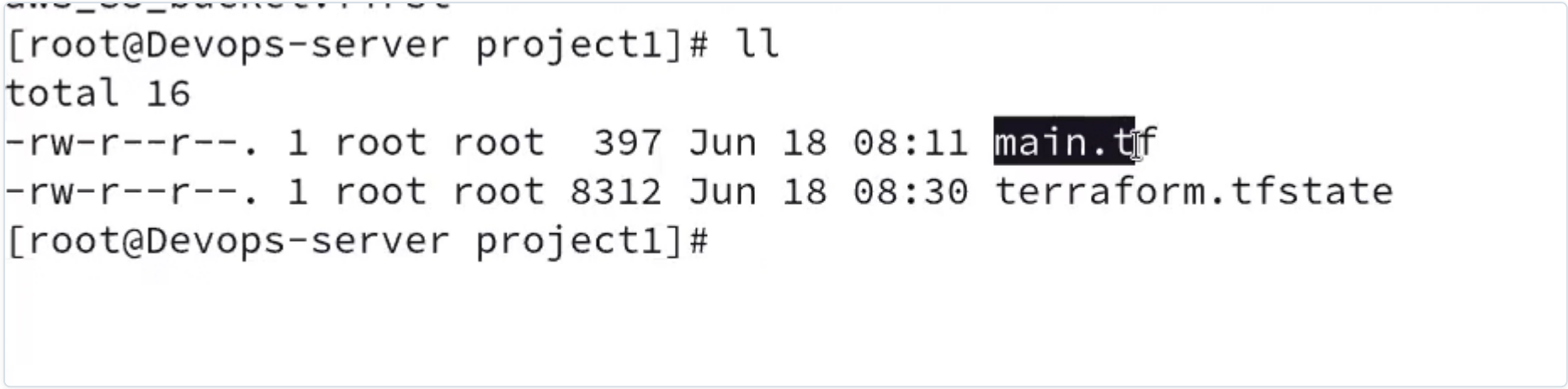
```
[root@Devops-server project1]# nl main.tf
     1  ########### Launch Ec2 Instance in Mumbai region
     2  resource "aws_instance" "linux" {
     3    ami           = "ami-0cca134ec43cf708f"
     4    instance_type = "t2.micro"
     5    key_name      = "ram_aws"
     6    tags = {
     7      Name = "Test_server"
     8      source = "terraform"
     9    }
    10  }

    11  ########### Launch S3 Bucket in Mumbai region
    12  resource "aws_s3_bucket" "first" {
    13    bucket = "linux23212google123"
    14    tags = {
    15          Name = "my_buck"
    16  }
    17  }
[root@Devops-server project1]#
```

06 / 25 Terminal — `terraform state list`: aws_instance.linux, aws_s3_bucket.first 23:14:14



07 / 25 Terminal — ll: main.tf + auto-created terraform.tfstate 23:15:16



08 / 25 Terminal — `terraform plan`: "No changes. Your infrastructure matches the configuration." 23:23:35

```
[root@Devops-server project1]# terraform plan
aws_s3_bucket.first: Refreshing state... [id=linux23212google123]
aws_instance.linux: Refreshing state... [id=i-0360dc4b6f1fe0859]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
[root@Devops-server project1]#
```

09 / 25 Desired State definition — "the state mentioned in your resource block of tf file" 23:25:37

Desired State and Current State

Desired State=> "this is the state that is mentioned in your resource block of tf file."

It is the State of infrastructure resources as per the terraform file (Eg. ex1.tf).

10 / 25 Current State definition — "the present state of the resources in the infrastructure" 23:26:27

Current State => It is the present state of the resources in the infrastructure. This may be different from desired state mentioned in .tf file.

If any difference found in the remote environment, based on state configuration, it will **update the remote environment object** like modify, create , delete etc. along with the state file also update about new identity of objects.

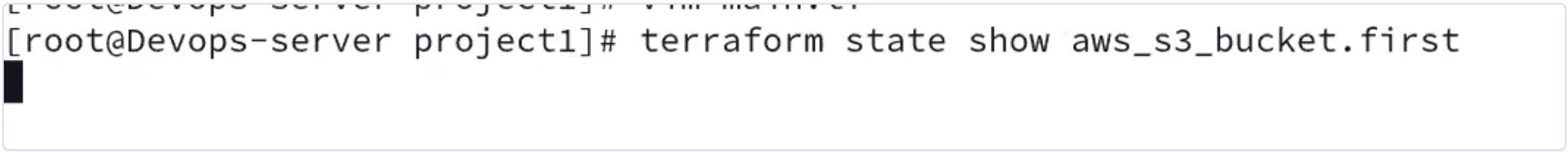
#terraform apply/plan

Tf config files => tf state file => Target Environment.

So finally in short, **Terraform is a state based engine.** When terraform runs it first looks at what you want (desired state) and scans what is in your cloud platform (actual state). Once it knows the desired state and the actual state, the next thing Terraform does is calculate a simple delta:

- If something exists in your desired state, but not in actual state **— it gets created.**
- If something exists and in both desired state and actual state, but is configured differently **— it gets updated.**
- If something exists in actual state but not in desired state **— it gets deleted.**

12 / 25 Terminal — `terraform state show aws_s3_bucket.first` (command entered) 23:34:15



13 / 25 Terminal — `terraform plan` diff after adding a Team=Linux tag: "1 to change" in-place update 23:36:34

```
Terraform will perform the following actions:

  # aws_s3_bucket.first will be updated in-place
  ~ resource "aws_s3_bucket" "first" {
        id                                   = "linux23212google123"
      ~ tags                                 = {
            "Name" = "my_buck"
          + "Team" = "Linux"
        }
      ~ tags_all                             = {
          + "Team" = "Linux"
            # (1 unchanged element hidden)
        }
        # (14 unchanged attributes hidden)

        # (3 unchanged blocks hidden)
    }

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.
```

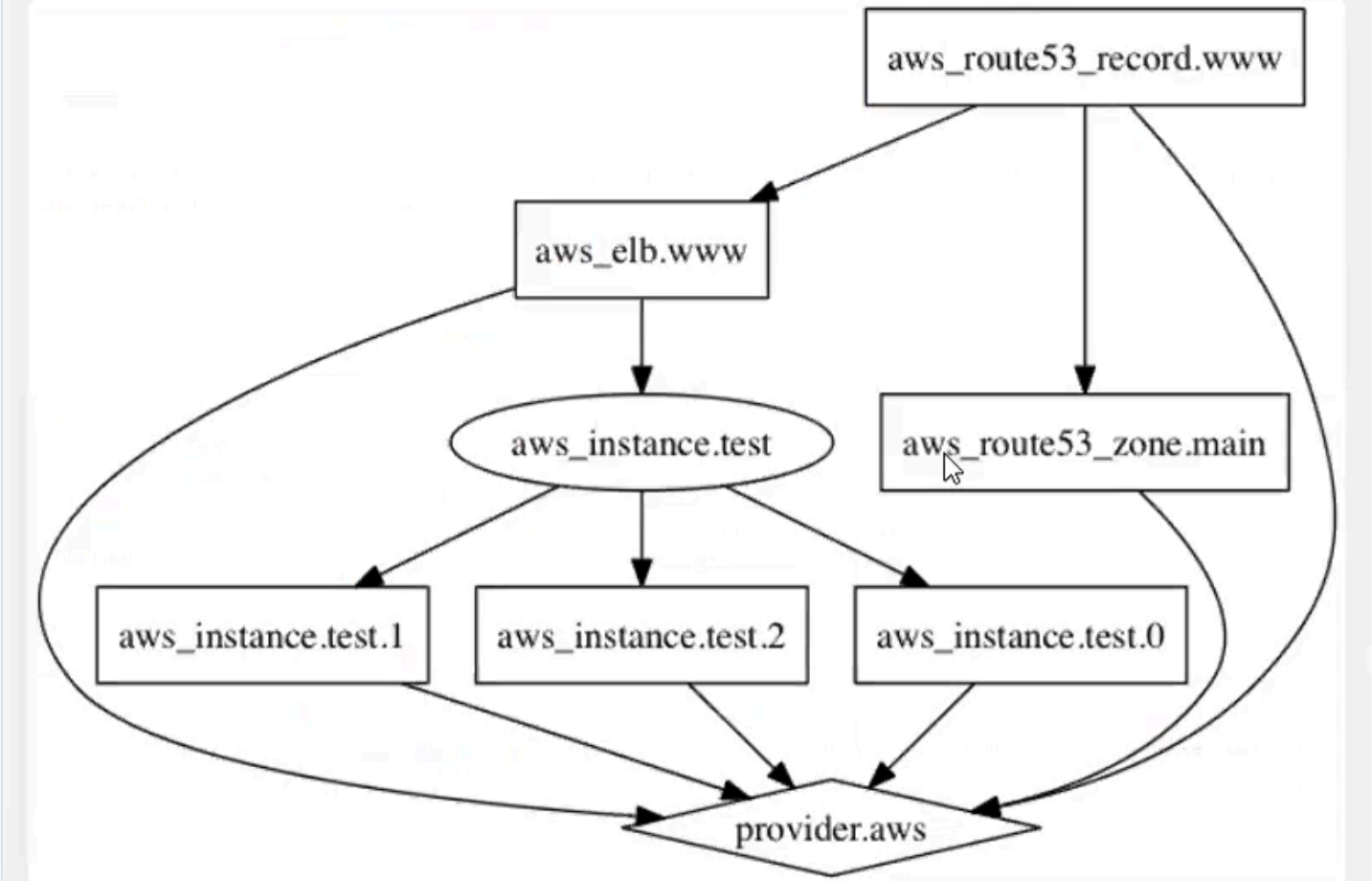
14 / 25 Terminal — `ll`: terraform.tfstate.backup auto-created after the apply 23:38:30

```
[root@Devops-server project1]# ll
total 28
-rw-r--r--. 1 root root  413 Jun 18 08:40 main.tf
-rw-r--r--. 1 root root 8374 Jun 18 08:43 terraform.tfstate
-rw-r--r--. 1 root root 8312 Jun 18 08:43 terraform.tfstate.backup
[root@Devops-server project1]#
```

15 / 25 Docs — `terraform graph -type=plan | dot -Tpng > graph.png` with an example rendered dependency graph 23:51:54

```
$ terraform graph -type=plan | dot -Tpng >graph.png
```

The following is an example result:



16 / 25 Code — aws_security_group "allow_ssh" + aws_vpc_security_group_ingress_rule referencing its .id 00:06:09

```
resource "aws_security_group" "allow_ssh" {
  name        = "my_sg"
  description = "Allow ssh traffic and all outbound traffic"

  tags = {
    Name = "allow_ssh"
  }
}

resource "aws_vpc_security_group_ingress_rule" "allow_ssh_ipv4" {
  security_group_id = aws_security_group.allow_ssh.id
  cidr_ipv4         = aws_vpc.main.cidr_block
  from_port         = 443
  ip_protocol       = "tcp"
  to_port           = 443
}

```

17 / 25 Terminal — apply output: security group created first, then the ingress rule (2 to add) 00:08:24



18 / 25 Code — ec2.tf: `vpc_security_group_ids = [aws_security_group.allow_ssh.id]` 00:17:57

```
[root@Devops-server project1]# nl ec2.tf
     1  resource "aws_instance" "linux" {
     2    ami           = "ami-0cca134ec43cf708f"
     3    instance_type = "t2.micro"
     4    key_name      = "deployer-key"
     5  vpc_security_group_ids = [aws_security_group.allow_ssh.id]
     6    tags = {
     7      Name = "Test_server"
     8      source = "terraform"
     9    }
    10  }

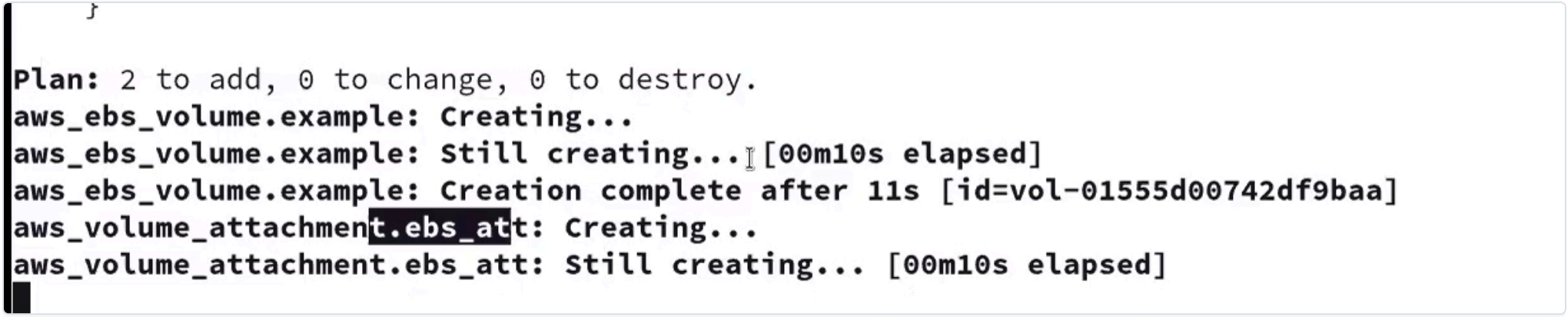
[root@Devops-server project1]# terraform a
```

19 / 25 Code — aws_volume_attachment "ebs_att" + aws_ebs_volume "example" referencing instance/volume .id 00:24:28

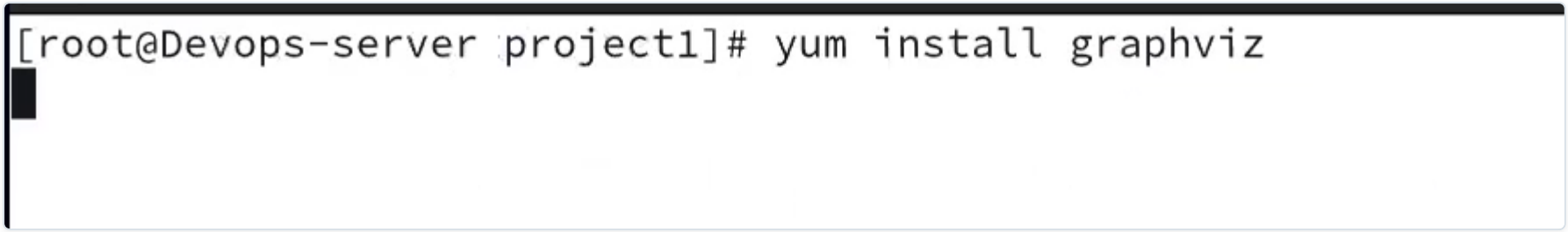
```
resource "aws_instance" "linux" {
  ami             = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name    = "deployer-key"
vpc_security_group_ids = [aws_security_group.allow_ssh.id]
  tags = {
    Name = "Test_server"
    source = "terraform"
  }
}
resource "aws_volume_attachment" "ebs_att" {
  device_name = "/dev/sdc"
  volume_id   = aws_ebs_volume.example.id
  instance_id = aws_instance.linux.id
}

resource "aws_ebs_volume" "example" {
  availability_zone = "ap-south-1b"
  size              = 5
}
~
```

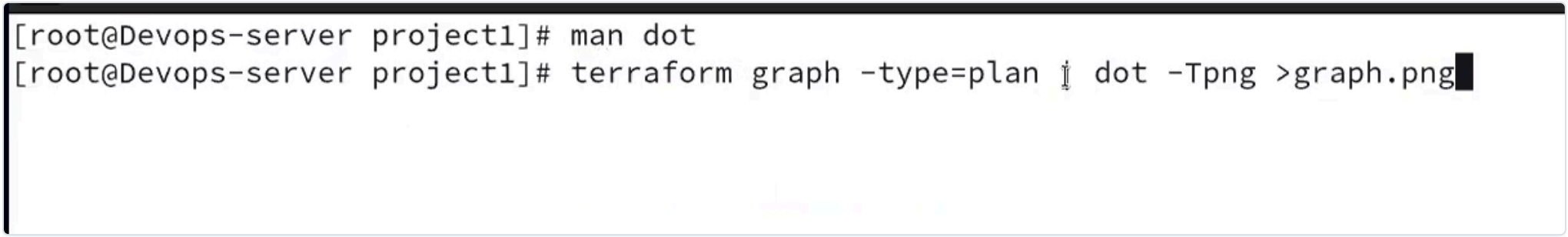
20 / 25 Terminal — apply output: EBS volume created, then volume attachment (2 to add) 00:26:00



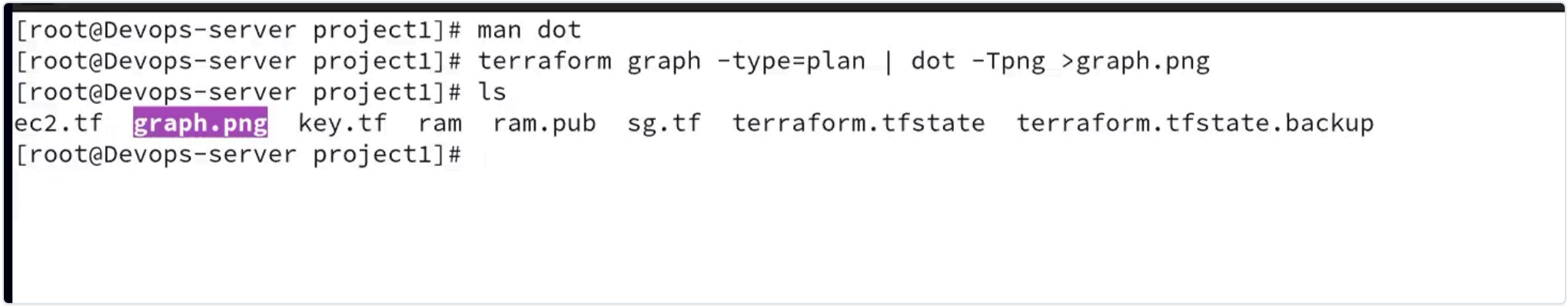
21 / 25 Terminal — `yum install graphviz` 00:28:54



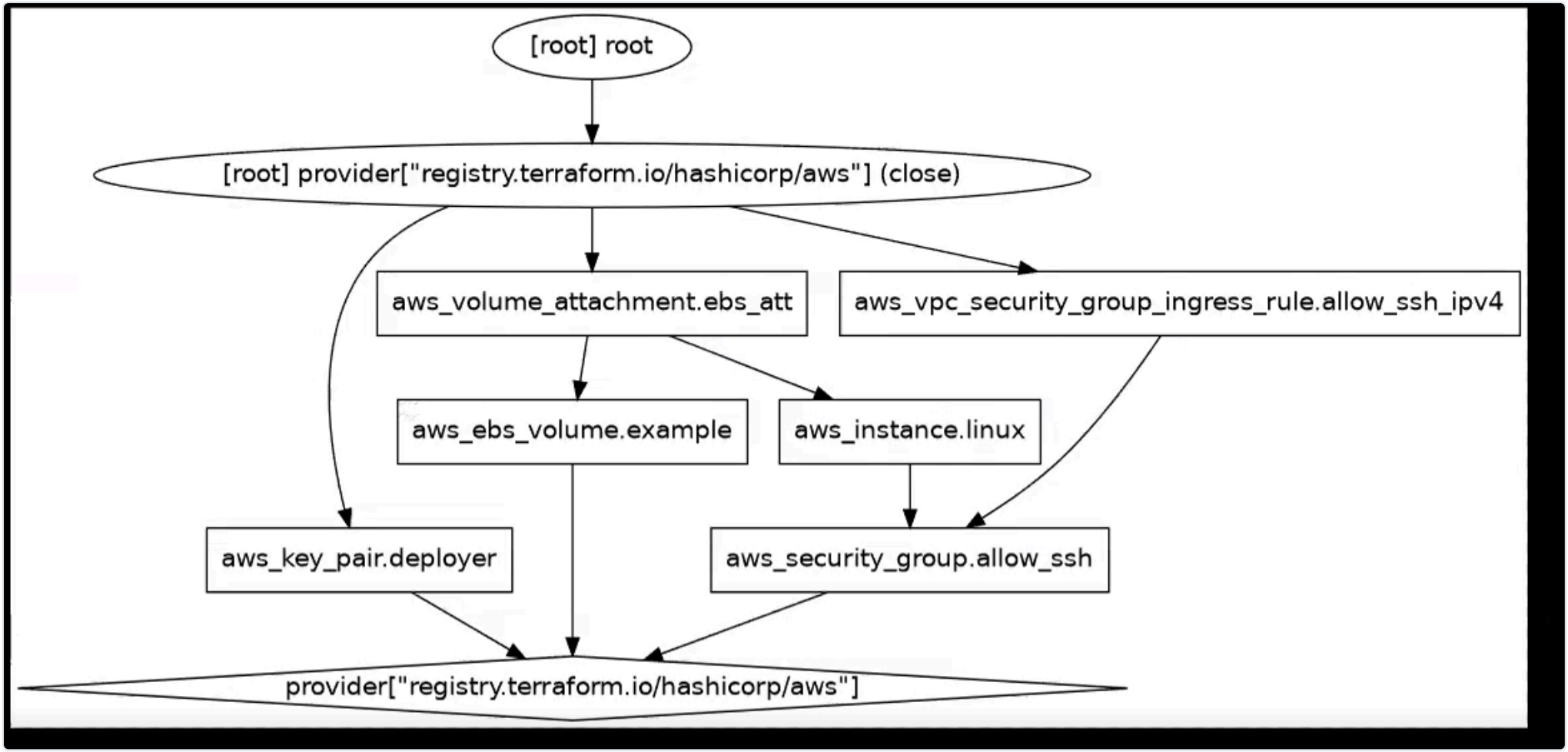
22 / 25 Terminal — `man dot; terraform graph -type=plan | dot -Tpng > graph.png` 00:29:50



23 / 25 Terminal — `ls`: graph.png generated alongside ec2.tf, key.tf, sg.tf, state files 00:30:39



24 / 25 Final graph.png — full dependency graph: key pair, security group, EBS volume, instance, volume attachment, ingress rule, all rooted at the AWS provider `00:31:36`



25 / 25 Bonus reference — terraform graph/taint/import/workspace/refresh/logging/provisioners/plugin cache/user data, one-line definitions 00:37:09

- terraform graph # **This** is used to generate a **visual representation of the resources** in your infrastructure and how they are **connected or depend** on each other.

- terraform taint # This is used to manually mark a specific resource as **tainted**. When a resource is tainted, it means Terraform will **consider it damaged or needing replacement** on the next apply operation.
- terraform import # This command used to **bring existing infrastructure** (that wasn't originally created with Terraform) under Terraform's management.
- terraform workspace # Terraform workspaces enable us to **manage multiple deployments** of the same configuration.
- terraform refresh # terraform refresh **reads the current settings** from all managed remote objects and updates the Terraform state to match.
- terraform logging # Terraform logging **records the internal steps** Terraform takes when managing your infrastructure.
- terraform provisioners # Terraform Provisioners are tools used within a Terraform configuration to **execute scripts or** commands on a local or remote machine as part of the resource provisioning process.

- terraform plugin cache # Terraform plugin cache is like a **local storage that saves plugins** to make Terraform faster and more reliable.

- terraform user data # Terraform user data is **kind of instructions** you give a new server to run automatically at startup.

Ram Misra